

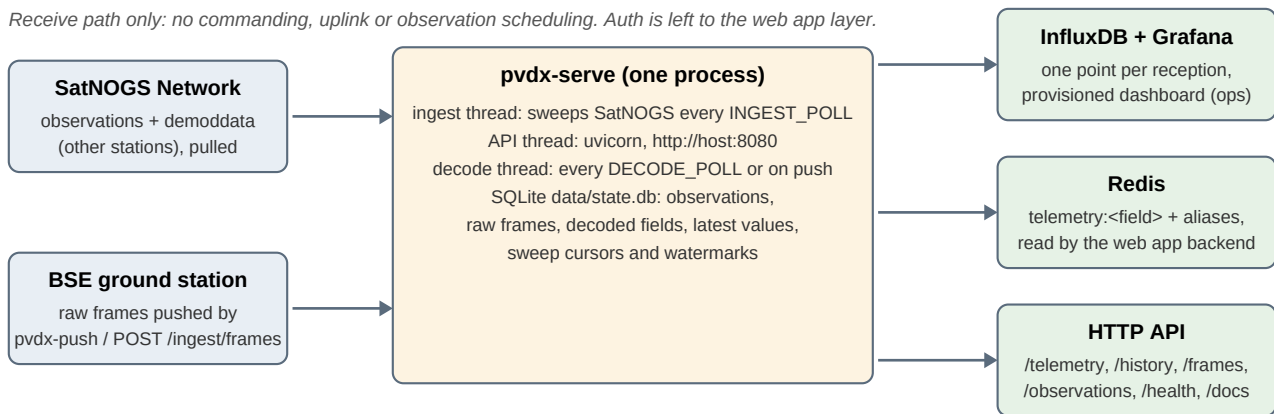
PVDX Ground Pipeline

Operations Guide: SatNOGS + ground-station ingest, decode, storage, HTTP API, Redis and dashboard

Brown Space Engineering · version 0.2.0 · 2026-10-04

This guide explains how to run and look after the cloud telemetry service: it pulls demodulated frames for one satellite from the SatNOGS Network, accepts frames pushed by our own ground station, stores them with provenance, decodes them, and serves the result to the mission-control web app (HTTP API and Redis) and to operators (Grafana). It assumes the repository at `/Users/bryan/code/bse_code` (adjust paths for another checkout). Until PVDX launches the service runs against a stand-in satellite; section 9 covers the swap.

Receive path only: no commanding, uplink or observation scheduling. Auth is left to the web app layer.



1. What runs where

A working installation is two things:

- **Three containers** started by Docker Compose: InfluxDB 2.7 on port 8086 (time series), Grafana 11.3 on port 3000 (dashboard, provisioned automatically with the datasource and the dashboard) and Redis 7 on port 6379 (the latest-value cache the web app backend reads). All three ports are published on 127.0.0.1 only (section 3).
- **The service** (pvdX-serve): one Python process with three parts. The **ingest** thread sweeps the SatNOGS observations list for the configured NORAD id and downloads every demodulated frame file into `data/state.db` (SQLite). The **decode** thread decodes new frames, writes one InfluxDB point per frame, keeps the decoded fields in SQLite and publishes the latest values to Redis. The **HTTP API** on port 8080 serves telemetry, history, frames, observations and health, and accepts frames pushed by our ground station (which are decoded within a second).

The service runs on your machine or, with the `cloud compose` profile, as a fourth container (section 3). The one-shot commands `pvdX-ingest` and `pvdX-decode` still exist for operations and are safe to re-run at any time: nothing is downloaded twice and nothing is skipped. Never run `pvdX-serve` and a polling `pvdX-ingest` against the same `data/state.db` at once. If no service is running, the dashboard is static and the Redis keys expire.

2. One-time setup

2.1 Requirements

- macOS or Linux with **Python 3.11 or newer** (uv recommended, plain venv/pip works).
- **Docker Desktop** with Compose v2 (docker compose version must work). Docker Desktop has to be running before docker compose up.
- A **SatNOGS Network API key**, optional but strongly recommended (see 2.3).

2.2 Install

```
cd /Users/bryan/code/bse_code
cp .env.example .env          # first time only; then edit .env (see 2.3 and 2.4):
                                # token, INFLUX_* secrets, INGEST_START, INGEST_TOKEN

uv venv --python 3.12 .venv
source .venv/bin/activate
uv pip install -e ".[dev]"    # or: pip install -e ".[dev]"
pytest                       # 151 tests, no network access needed
```

Every command in this guide assumes the virtual environment is active (source .venv/bin/activate) and the working directory is the repository root, because .env and data/ are resolved relative to it.

2.3 The SatNOGS token

SatNOGS **Network** and SatNOGS **DB** issue separate tokens. The pipeline needs the Network one, shown under "API Key" at <https://network.satnogs.org/users/edit/>. A DB token is rejected by the Network API with 401 Invalid token; pvdX-ingest logs that once and continues anonymously, because reads work without a token. The difference is speed: the observations list is throttled to **60 requests per hour anonymously and 240 per hour with a valid token**, and each request returns 25 observations. The token is only ever sent to the SatNOGS host, never to the object storage that serves frame files.

2.4 Configuration reference (.env)

Variable	Default	Meaning
SATNOGS_API_TOKEN	(empty)	SatNOGS Network API key. Empty means anonymous access.
SATNOGS_NETWORK_URL	https://network.satnogs.org	API base URL.
NORAD_CAT_ID	(required)	Satellite to ingest. SatNOGS temporary ids (98xxx/99xxx) also work.
INGEST_START	now minus 7 days if unset (.env.example pins a fixed date)	Floor for every sweep: a sweep starts at max(INGEST_START, watermark minus overlap), so in practice only the first sweep is affected. Keep it a fixed date a few days back; an old date triggers a multi-week first backfill.
INGEST_OVERLAP_HOURS	48	How far behind the watermark each later sweep re-scans.
INGEST_STATUS	good	Observation status filter; frames only exist on good observations.
INGEST_MIN_INTERVAL	from throttle	Seconds between list requests (63 anonymous, 15.8 with token). At least 15, the authenticated throttle; a lower value is a configuration error (exit 2).

Variable	Default	Meaning
STATE_DB	data/state.db	SQLite file with observations, raw frames and sweep state.
DECODER	(empty)	satnogs:<struct> (e.g. satnogs:crocube) or pvdX.
INFLUX_URL / ORG / BUCKET	http://localhost:8086 / bse / telemetry	InfluxDB connection used by pvdX-decode. INFLUX_URL must start with http:// or https://. ORG and BUCKET (not URL) are also read by docker compose to initialise the container. Keep the bucket named telemetry: the dashboard JSON hardcodes bucket and measurement names.
INFLUX_TOKEN	(set one)	Admin token you choose; the InfluxDB container registers it on its first start and pvdX-decode and Grafana authenticate with it.
INFLUX_USERNAME / PASSWORD	admin / (set one, 8+ characters)	InfluxDB UI login at localhost:8086. InfluxDB refuses shorter passwords on first start; the container then exits and Grafana never starts.
INFLUX_RETENTION	0 (forever)	Retention of the telemetry bucket, applied by docker compose when the InfluxDB volume is first initialised (e.g. 30d). Not read by the Python commands.
GRAFANA_ADMIN_PASSWORD	admin	Grafana login (user admin). Applied only when the grafana-data volume is first created; later, change it in the Grafana UI or with grafana cli admin reset-admin-password.
GRAFANA_BIND / API_BIND	127.0.0.1 / 127.0.0.1	Host address docker compose publishes Grafana and the pvdX container's API on. 0.0.0.0 only behind a TLS reverse proxy, or with a network/cloud firewall or security group in front; ufw does not see Docker-published ports (section 3).
API_HOST / API_PORT	127.0.0.1 / 8080	Where pvdX-serve listens. 0.0.0.0 inside a container; for a host-run pvdX-serve it decides whether other machines can reach the API. API_PORT is also the host port of the pvdX container.
API_CORS_ORIGINS	http://localhost:3000	Browser origins allowed to call the API directly (comma separated, or *).
INGEST_TOKEN	(empty)	Shared secret the ground station must send as X-Ingest-Token on POST /ingest/frames; checked in constant time before the request body is read. Empty accepts pushes from anyone (a warning is logged).
INGEST_POLL / DECODE_POLL	600 / 60	Seconds between SatNOGS sweeps and between decode passes in pvdX-serve. Pushed frames are decoded at once.
TELEMETRY_STALE_AFTER	3600	GET /telemetry reports stale=true when the newest decoded frame is older than this many seconds.
TELEMETRY_ALIASES	(empty)	Extra names for decoded fields, comma separated, each alias=field[*scale][[+ -]offset[:unit]; the alias value is raw * scale + offset (section 7.2). .env.example maps the web app's battery/temperature/signal_rssi/uptime_seconds to CroCube fields in V, degC, dBm and s.
REDIS_URL	(empty = off)	redis://localhost:6379/0 for the compose Redis; must start with redis://, rediss:// or unix://. Latest values are published at startup and after every decode pass that writes at least one decoded frame to InfluxDB.

Variable	Default	Meaning
REDIS_KEY_PREFIX / REDIS_TELEMETRY_TTL	telemetry: / 3600	Key prefix (the web app reads telemetry:<name>) and lifetime of the keys, counted from the last publish; expired keys make the web app show stale.
PUSH_URL / PUSH_STATION / PUSH_TOKEN	(empty)	Only for pvdx-push on the ground station computer: service URL, this station's name, and the service's INGEST_TOKEN.

Any pvdx-serve, pvdx-ingest, pvdx-decode or pvdx-push flag overrides the file for that run (section 5). An invalid value (a malformed alias, a URL with the wrong scheme, an interval below its minimum) stops every command at startup with configuration error: ... and exit code 2. The InfluxDB credentials and the Grafana admin password are baked into the containers' data volumes on first start; if you change them later, wipe the volumes (section 8.5) or the containers will keep the old ones.

3. Starting and stopping the stack

```

docker compose up -d          # start InfluxDB + Grafana + Redis (pulls images once)
docker compose ps             # all three Up and (healthy)
open http://localhost:3000    # login admin / GRAFANA_ADMIN_PASSWORD; click "Skip"
                              # on the "Update your password" prompt (or set a new one)

docker compose down           # stop; data volumes are kept
docker compose down -v        # stop AND delete all InfluxDB/Grafana data

docker compose --profile cloud up -d --build # ALSO build + run the pvdx container
docker compose --profile cloud logs -f pvdx  # its log; state in ./data, API on :8080
docker compose --profile cloud down          # stop everything including the service

```

The pvdx container reads .env, replaces the InfluxDB and Redis URLs with the compose service names and mounts ./data for the state database. Use it or a host-run pvdx-serve, never both on the same data/state.db. All four containers restart unless stopped, so they come back after a reboot; Docker restarts a container when its process exits, never because it is merely (unhealthy).

Network exposure. Every port is published on 127.0.0.1 by default: InfluxDB and Redis always (Redis has no password), Grafana and the API unless GRAFANA_BIND / API_BIND in .env say otherwise. Docker writes its own iptables rules for published ports, so they bypass host firewalls such as ufw; the bind address is the control that holds. A host-run pvdx-serve listens on API_HOST instead (default 127.0.0.1), where ufw applies as usual. On a cloud host put a TLS reverse proxy (Caddy, nginx) in front of the API and Grafana; a proxy on the same host reaches them on 127.0.0.1. Set GRAFANA_BIND=0.0.0.0 or API_BIND=0.0.0.0 only when the proxy runs elsewhere and a network/cloud firewall or security group (not ufw on the host) limits who can reach those ports. Before Grafana is reachable by anyone else, change its admin password (GRAFANA_ADMIN_PASSWORD only applies to a new grafana-data volume; otherwise use the Grafana UI or docker compose exec grafana grafana cli admin reset-admin-password <new>) and set INGEST_TOKEN.

The provisioned dashboard is at http://localhost:3000/d/pvdx-telemetry in the **PVDX** folder. Grafana re-reads grafana/dashboards/*.json every 30 seconds, so edits to the JSON file appear without a restart. InfluxDB's own UI at http://localhost:8086 is available for ad-hoc Flux queries.

4. Running the service

4.1 The service: pvdx-serve

```
pvdx-serve                # ingest every INGEST_POLL s, decode every DECODE_POLL s
                           # (or on push), publish to Redis, API on
                           # http://127.0.0.1:8080 (docs at /docs). Ctrl-C stops it.
pvdx-serve --no-ingest    # frames arrive by push only (no SatNOGS sweeps)
pvdx-serve --port 9000 -v # other port; debug log + HTTP access log
curl -s localhost:8080/health # ok/degraded, workers, InfluxDB and Redis reachability
```

Start the Docker stack first. The decode part verifies the InfluxDB token, org and bucket at startup and keeps retrying if they are wrong or unreachable. If Redis is down, the failed publish is logged as a warning and decoding goes on; the keys are written again by the next decode pass that writes at least one decoded frame to InfluxDB, or at the next start. The first start after an upgrade migrates data/state.db in place (from 0.1: to schema 2, re-decoding the frames that were already decoded, then on to 3; from 0.2.0: to schema 3, grouping duplicate receptions). Back it up first and see 8.8 for the InfluxDB points. Ctrl-C and SIGTERM (docker stop, kill, systemd) stop it the same way: the API first, then the threads after their current step; a sweep in progress resumes on the next start.

A restart policy (launchd/systemd restart-on-failure, the containers' unless-stopped) only acts when the process exits. A worker thread that dies leaves the process and the API running: GET /health then reports degraded with that worker stopped, still with HTTP 200, so watch the status field and restart pvdx-serve by hand after fixing the cause. Only with --no-api does the process exit (code 1) once every worker has stopped.

4.2 One-shot commands

```
pvdx-ingest                # one sweep: new observations + frames -> data/state.db
pvdx-decode                # decode new frames -> InfluxDB (+ Redis if REDIS_URL)
pvdx-ingest --stats        # counters: observations, frames, decoded, watermark
pvdx-ingest --poll 600     # the two loops as processes, instead of pvdx-serve
pvdx-decode --poll 60
```

These are the same code paths the service runs; use them for a single sweep, a re-decode, or on a machine without the API. Stop them with Ctrl-C, which is safe at any moment.

4.3 What to expect on the first run

- The first sweep covers INGEST_START to now. Each API page holds 25 observations and the client waits about 63 s between pages anonymously (16 s with a token), so a week of an active satellite (several hundred observations) takes tens of minutes of paging.
- Frames are downloaded after every page, so data flows early. The stand-in satellite delivers up to ~100 frames per pass; at ~0.25 s per file a busy page can take 10 minutes to download.
- Use --max-pages N to ration a run. A run stopped by --max-pages, Ctrl-C or a crash leaves a resumable sweep; the next run continues from the saved cursor and does not re-fetch stored pages.
- When a sweep reaches its last page it is marked complete and the satellite's watermark advances to the time that sweep was opened. Later sweeps start at watermark minus INGEST_OVERLAP_HOURS (but never earlier than INGEST_START), which picks up observations vetted late and frames uploaded late.
- Check INGEST_START before the first run: at one page per minute plus frame downloads, a multi-week backfill of an active satellite takes hours. A few days is plenty for a first look.

4.4 Reading the log lines

```
migrating data/state.db from schema version 2 to 3          (once, after an upgrade)
migration done: 15926 stored frame(s) grouped into 7406 transmission(s),
                  8520 duplicate reception(s)
starting sweep 3 for NORAD 62394: status=good since 2026-09-24T12:00:00Z (anonymous)
page 1: 25 observation(s), 4 new, 380 new frame(s) queued
--max-pages 1 reached; sweep 3 will resume next run
sweep 3 for NORAD 62394 paused (will resume): 1 page(s), 25 observations (4 new), ...
decode: 2433 frame(s) read, 91 decoded (40 duplicate reception(s)), 91 written,
       0 rejected by InfluxDB, 0 empty, 2342 not telemetry/undecodable,
       79 Redis key(s) published
push from BSE Providence: 2 frame(s) stored, 0 duplicate(s) (NORAD 62394)
decode: 2 frame(s) read, 2 decoded (0 duplicate reception(s)), 2 written, ...
       79 Redis key(s) published
```

A large "not telemetry/undecodable" count is normal for CroCube: image-download passes produce hundreds of 160-byte file chunks per observation that are not telemetry (SatNOGS DB does not decode them either). Only the AX.25 beacons (OBC, PSU, UHF, ...) become points. "Duplicate reception(s)" counts decoded frames that are copies of a transmission another frame is the primary of (section 7.3); they are written to InfluxDB with primary=false. The "push from" line counts resent frames as duplicates, which is a different thing: the same push received twice.

5. Command reference

5.1 pvdX-serve

Flag	Effect
--env PATH	dotenv file to load (default .env).
--host / --port	Override API_HOST / API_PORT.
--ingest-poll S / --decode-poll S	Override INGEST_POLL / DECODE_POLL.
--measurement NAME	InfluxDB measurement (default telemetry).
--no-ingest	Do not sweep SatNOGS; frames arrive by push only. NORAD_CAT_ID is then optional.
--no-decode	API and ingest only (no InfluxDB, no Redis).
--no-api	Ingest and decode only.
-v	Debug logging plus the HTTP access log.

5.2 pvdX-push (ground station computer)

Flag / argument	Effect
PATH ...	Frame files (raw bytes, one frame each) or directories of them (one level deep).
--url / --station / --token	Override PUSH_URL / PUSH_STATION / PUSH_TOKEN.
--norad ID	Satellite (default NORAD_CAT_ID).
--time ISO	Receive time for every file; default is the file's modification time.
--frequency HZ	Downlink frequency recorded with every frame.
--meta KEY=VALUE	Extra metadata (repeatable); the file name is always recorded.
--stdin	Read '[ISO-time] hex' lines from stdin instead of files (time defaults to now).
--watch SECONDS	Rescan the given directories forever, sending files not yet sent.

Exit codes: 0 sent, 1 the service rejected the frames or was unreachable after 3 retries, 2 configuration error. 429/5xx/transport errors are retried with backoff; resending a frame is harmless (the service reports it as a duplicate).

5.3 pvdX-ingest

Flag	Effect
--env PATH	dotenv file to load (default .env).
--norad ID	Override NORAD_CAT_ID.
--since ISO	Override INGEST_START for the first sweep of a satellite (no effect once a watermark exists unless it is later than watermark minus overlap).
--state PATH	Override STATE_DB.
--status good bad unknown future failed	Override INGEST_STATUS.
--overlap-hours H	Override INGEST_OVERLAP_HOURS.
--min-interval S	Seconds between list requests (default from the throttle; at least 15).
--max-pages N	Stop after N pages this run; the sweep resumes next run.
--poll SECONDS	Loop forever, sleeping SECONDS between runs.
--retry-failed	Re-queue frames that exhausted their 5 download attempts.
--stats	Print state-database counters as JSON and exit.
-v	Debug logging (pacing, per-frame lines).

5.4 pvdX-decode

Flag	Effect
--env PATH	dotenv file to load (default .env).
--decoder SPEC	Override DECODER, e.g. satnogs:crocube, satnogs:canvas, pvdX.
--norad ID	Only decode this satellite's frames (default NORAD_CAT_ID; empty means all).

Flag	Effect
--state PATH	Override STATE_DB.
--measurement NAME	InfluxDB measurement (default telemetry). The dashboard expects telemetry.
--limit N	Decode at most N frames this run.
--redo	Re-decode frames already marked decoded (after a decoder change, or to retry frames marked error). With NORAD_CAT_ID set, only that satellite's frames. Primary assignments are kept.
--dry-run	Decode without writing, marking, assigning primaries or publishing anything (a schema-2 database is still migrated on open, which assigns every primary); prints the first 3 decoded frames of each 500-frame batch as JSON plus the counters.
--poll SECONDS	Loop forever, sleeping SECONDS between runs.
-v	Debug logging (per-frame decode errors).

Exit codes: 0 success, 1 a run finished with errors (frame download failures, InfluxDB unreachable), 2 configuration error, 130 interrupted with Ctrl-C.

6. Viewing output

6.1 The dashboard

Open <http://localhost:3000/d/pvdx-telemetry>. Variables along the top select the NORAD id (discovered from the data), ground stations, which receptions to show, and which decoded fields to plot; the voltage and temperature panels are pre-filtered by name pattern. Panels: frames, receptions, observations and stations in range; last frame time; frames per hour; battery/bus voltages; temperatures; any field; frames by station; and a table with the latest value of every field with its observation id and station. Values are the decoder's raw units (CroCube: battery in millivolts, temperatures in hundredths of a degree C, RSSI dBm = raw/2 - 134); the alias conversion of section 7.2 does not reach InfluxDB. The default time range is the last 7 days with auto-refresh every minute; widen it if the last frame is older.

The **Receptions** variable decides how duplicate receptions are shown (section 7.3). "Primary only" (the default) keeps one point per transmission, "Copies only" the other receptions, "All" every reception. It applies to the telemetry panels, "Frames in range", "Frames per hour" and the latest-values table. With a station selected and "Primary only", those panels show only the transmissions this station supplied the primary for; choose "All" to see everything it heard. "Receptions in range" and "Frames by station" always count every reception (one SatNOGS station often twice: its own file and the _gN file), and the observation and station counters and "Last frame received" ignore the variable.

The ground-station picker lists the SatNOGS station ids of the ground_station tag. Frames pushed by our own station have no such tag: they are included under All and appear by name in "Frames by station" and the latest-values table, but cannot be selected and disappear as soon as specific stations are picked. The observation and station counters count all pushed frames as one observation and one station.

The dashboard is decoder-agnostic: it discovers field names from InfluxDB, so it works unchanged after the PVDX decoder replaces the stand-in. Adjust the regexes of the voltage_fields and temperature_fields variables in grafana/dashboards/pvdx-telemetry.json if PVDX field names use other words.

6.2 The HTTP API

Interactive documentation is at <http://127.0.0.1:8080/docs>. Times are ISO-8601 UTC; since and until also accept relative forms such as -6h or -2d. On the /telemetry endpoints norad defaults to NORAD_CAT_ID, or to the only satellite in the database; /frames and /observations without norad list every satellite.

Endpoint	Returns
GET /health	ok or degraded; per-satellite counters, watermark and last sweep; worker threads; InfluxDB and Redis reachability.
GET /telemetry	{telemetry: {field: value, alias: value, ...}, units: {alias: unit}, stale, norad_cat_id, frame_time, source}; the flat shape the mission-control backend serves today. Field values raw, alias values converted (section 7.2).
GET /telemetry/latest	Every field with its raw value, frame_time, frame_id, source (satnogs/groundstation) and station_name, plus staleness and the alias map (alias to field).
GET /telemetry/fields	Known field names, type and when each was last seen.
GET /telemetry/history?field=&since=&until=&source=&copies=&limit=	One field over time, oldest first (the newest limit points); one point per transmission unless copies=true (section 7.3). field may be an alias: values converted, unit set. The response names the stored_field it read.
GET /frames?norad=&since=&until=&source=&station=&decode_status=&primary=&limit=&offset=	Frame metadata with family, primary_frame_id and primary, newest first. primary=true lists primaries, primary=false copies and frames not yet assigned.
GET /frames/{id}	One frame: raw_base64, raw_hex, decoded fields, push metadata, observation summary, family and primary_frame_id.
GET /observations?norad=&since=&until=&ground_station=&limit=&offset=	SatNOGS observations with per-observation frame counts.
GET /observations/{id}	One observation including the complete SatNOGS record.
POST /ingest/frames	Store frames received by our ground station (section 8.7).

```
curl -s 'http://127.0.0.1:8080/telemetry' | jq .telemetry.battery
curl -s 'http://127.0.0.1:8080/telemetry/history?field=psu_battery&since=-3d' \
  | jq '.points[-1]'
curl -s 'http://127.0.0.1:8080/frames?source=groundstation&limit=5' \
  | jq '.frames[] | {id, frame_time, decode_status}'
curl -s 'http://127.0.0.1:8080/telemetry/history?field=battery&since=-1d' \
  | jq '{stored_field, unit, count}'
```

Bad parameters get 400 or 422, not a 500. 400: a time that is neither ISO-8601 nor relative or lies outside the years 1-9999, an invalid field name, or a /telemetry endpoint called without norad while NORAD_CAT_ID is unset and the database does not hold exactly one satellite. 422: a value outside its range: ids (/frames/{id}, /observations/{id}) and ground_station must be at least 1, norad 1 to 999 999 999, field 1 to 200 characters long, limit/offset within their bounds, and source, decode_status, primary and copies one of their documented values. An id that does not exist gives 404. Every request opens its own SQLite connection, so concurrent requests are safe.

6.3 Without Grafana or the API

```
pvdX-ingest --stats                                # JSON counters and the watermark
pvdX-decode --dry-run --redo --limit 100           # decodes 100 frames; prints first 3 as JSON
sqlite3 data/state.db "select id, source, family, primary_frame_id, frame_time,
station_name, decode_status from frames order by frame_time desc limit 10"
sqlite3 data/state.db "select field, value, frame_time, source from latest_values"
```

Raw frame bytes never leave SQLite: `frames.raw` holds them with source (satnogs or groundstation), sha256, size, the frame time (from the SatNOGS file name or the pushed receive time), push metadata, download/decode bookkeeping, the decoder name, the decoded fields as JSON, the family (native, grsat or push) and the `primary_frame_id` (section 7.3). `latest_values` keeps the newest value of every field per satellite, from primaries only, and only moves forward in frame time. `observations` holds station id, name, position, start/end, observer, transmitter, frequency, TLE lines and the complete API record as JSON.

7. Data model in InfluxDB and Redis

7.1 InfluxDB points

Item	Value
Bucket / measurement	telemetry / telemetry (both hardcoded as constant variables in the dashboard JSON)
Tags	<code>norad_cat_id</code> , <code>sat_id</code> , <code>decoder</code> , <code>source</code> (satnogs/groundstation), <code>observation_id</code> , <code>ground_station</code> , <code>station_name</code> , and <code>primary</code> (true for the primary reception of a transmission, false for its copies; section 7.3). Pushed frames have no observation, so their points carry no <code>sat_id</code> , <code>observation_id</code> or <code>ground_station</code> tag at all (absent, not empty).
Fields	every decoded value in the decoder's raw units (numbers as floats, strings as strings) plus <code>frame_id</code>
Time	frame time from the SatNOGS file name or the pushed receive time (fallback: observation start), nanosecond precision with the frame's <code>state.db</code> row id as sub-second offset so same-second frames stay distinct

Every reception gets its own point; only the `primary` tag tells copies apart. Re-writing a frame overwrites its point only while the tag set and the row id are unchanged (a frame's primary never changes, so neither does its primary tag). A tag-set change (a new tag, such as `primary` for points written by 0.2.0, or a renamed decoder) or a rebuilt `state.db` (new row ids) writes duplicate points, so delete the measurement (8.5) before re-decoding in those cases.

A frame is marked decoded only after its point is written. An InfluxDB outage (5xx, connection refused) or a credential/bucket problem (401/403/404) leaves the batch undecoded to be retried on the next run; a point InfluxDB refuses because of its data (400/422, for example a field type conflict) is marked error with the reason and does not block the rest of the batch. So is a frame that makes the decoder raise anything other than a normal rejection (`decode_error` reads `<decoder>: unexpected <exception>: ...`, with an ERROR log line per frame and one traceback per pass) and a frame with neither a frame time nor an observation (no InfluxDB point: ...): none of them blocks the frames behind it. Fix the decoder, then `pvdX-decode --redo`.

7.2 Redis keys and telemetry aliases

Redis (when REDIS_URL is set) is the telemetry half of GS3 in the comms design: a latest-value cache for the web app backend (there is no command queue). Values come from latest_values, that is from primaries only. All keys carry the same TTL:

Key	Value
telemetry:<field>	Newest raw value of each decoded field as a string (7933, SAFE).
telemetry:<alias>	The value of an aliased field under its TELEMETRY_ALIASES name, converted when the alias has a scale or offset (battery = 7.933).
telemetry:_meta	JSON: satellite, newest frame time/id/source/station, published_at, aliases (alias to field), units (alias to unit), TTL.
telemetry:_all	JSON: every field with its raw value, frame_time, frame_id, source and station_name.

The keys are rewritten after every decode pass that writes at least one decoded frame to InfluxDB, and once at startup from latest_values. The TTL counts from that publish, not from the frame time, so a restart or a pass that decodes only old, backfilled frames (or only duplicate copies) re-arms it on values that may be hours or days old. A Redis outage loses nothing (latest_values keeps every value): the failed publish is logged, and keys that expired or were lost meanwhile come back only with the next decode pass that writes a decoded frame, or on a restart.

TELEMETRY_ALIASES names fields for the web app and can convert them to engineering units. Each comma-separated entry is alias=field[*scale][(+|-)offset][:unit]: the alias value is raw * scale + offset (rounded to 12 significant digits) and the unit is a free-form label; a plain alias=field passes the raw value through, strings included. The CroCube example in .env.example:

```
TELEMETRY_ALIASES=battery=psu_battery*0.001:V,temperature=obc_temp_mcu*0.01:degC,
                  signal_rssi=uhf_act_rssi_raw*0.5-134:dBm,uptime_seconds=obc_uptime:s
(one line in .env; wrapped here)
psu_battery 7933 (mV)          -> battery 7.933 (V)
obc_temp_mcu 163 (1/100 degC) -> temperature 1.63 (degC)
uhf_act_rssi_raw 83           -> signal_rssi -92.5 (dBm)
obc_uptime 21345256 (s)       -> uptime_seconds 21345256 (s), passed through
```

Only the aliases are converted, and only in the telemetry:<alias> keys, the alias values of GET /telemetry and GET /telemetry/history?field=<alias>; units appear in units in telemetry:_meta and GET /telemetry, and as unit in the history response. Everything else stays raw: the telemetry:<field> keys, telemetry:_all, /telemetry/latest, /frames, SQLite, InfluxDB and Grafana. A converting alias whose field holds a string is logged once and left out of Redis (null in the API). signal_rssi is the RSSI measured by the **spacecraft's** UHF receiver, not the downlink strength at a ground station; the RSSI our station sends with a push is kept only in that frame's meta.rssi.

7.3 Duplicate receptions

A **reception** is one stored frame; a **transmission** is one frame the satellite sent. One transmission often arrives several times: several SatNOGS stations hear the pass, SatNOGS publishes a gr-satellites_gn file next to each station's own decoder output, and our ground station may push the same bytes. In a 15,926-frame CroCube database 8,520 receptions (53.5%) are copies, and so are 402 of its 895 decoded frames. Every reception is kept and decoded, and every decoded one is written to InfluxDB; each transmission has one **primary** reception, and the other receptions point at it (frames.primary_frame_id).

Family	Which frames	Time stamp
native	SatNOGS file from the station's own gr-satnogs decoder (plain or _N name)	station UTC clock when the frame was written: reception plus decoder latency
grsat	SatNOGS _gN file written by gr-satellites	0-16 s early: counted from a start time the SatNOGS client takes before launching gr-satellites (constant within an observation, different per station)
push	frame pushed by our ground station	the received_at the station reports (UTC)

- **Window rule.** Copies have the same satellite and identical bytes (sha256), and their effective time (the frame time, else the observation start) lies within 30 s of the primary when both come from one SatNOGS observation, within 45 s otherwise (station clocks have been seen up to 39 s late). The window is anchored on the primary, never chained. In the recorded CroCube data identical bytes recur at least 221 s apart, so a repeat is a new transmission.
- **Choosing the primary.** At the start of every decode pass (not in --dry-run, although the migration of a schema-2 database on open assigns every primary even then), each frame read that has no primary yet gets one. Native and pushed frames are placed before gr-satellites copies, so they become the primary when they are read in the same pass; each frame joins the nearest primary in its window or becomes a primary itself. Primaries therefore prefer native time stamps.
- **Sticky.** An assignment never changes. A _gN copy that was assigned before its native twin was downloaded stays the primary, with its early time; so do transmissions heard only as _gN (7 of 493 decoded transmissions in that database).
- **What uses primaries.** latest_values (and with it /telemetry, /telemetry/latest and Redis), the default /telemetry/history (copies=true returns every reception, each point marked primary; with source= the default keeps only primaries from that source) and, by default, the dashboard (section 6.1). /frames shows family, primary_frame_id and primary and filters with primary=true|false.

Databases from 0.2.0 (schema 2) get the two columns and their primaries automatically on first open, in one transaction; the InfluxDB points must then be rebuilt once (8.8).

8. Routine operations

8.1 Change the satellite

```
# .env
NORAD_CAT_ID=68635
DECODER=satnogs:canvas
# then simply run pvdx-ingest / pvdx-decode; state and watermarks are per NORAD id
```

Any satellite with a struct in the satnogs-decoders package works; the DB name of the decoder (as shown on <https://db.satnogs.org>) is the struct name, case-insensitive. The survey table in README.md lists the most active candidates measured on 2026-09-26.

8.2 Re-decode after a decoder change

```
pvdx-decode --redo # re-decode NORAD_CAT_ID's frames, overwriting points
# Switching to a different DECODER name? Delete the old points first (8.5):
# "decoder" is a tag, so the dashboard would otherwise show both series.
```

8.3 Recover frames that failed to download

```
pvdX-ingest --stats          # frames_failed > 0 ?
pvdX-ingest --retry-failed   # resets their attempt budget, then runs a normal sweep
```

8.4 Rotate the SatNOGS token

Edit SATNOGS_API_TOKEN in .env and restart pvdX-ingest. A rejected token shows up once as an ERROR line and the run continues anonymously. If the variable is also exported in your shell, unset it or open a new terminal: an exported environment variable always wins over .env.

8.5 Reset everything

```
# stop any running pvdX-ingest / pvdX-decode first (Ctrl-C)
docker compose down -v          # deletes InfluxDB + Grafana data (and their credentials)
rm -f data/state.db data/state.db-wal data/state.db-shm
docker compose up -d
pvdX-ingest                      # exit code 1 only means some frame downloads failed
pvdX-decode
```

To clear only the InfluxDB points but keep the SQLite frames, stop pvdX-serve, delete the measurement, re-decode and start the service again. Never source .env in your working shell: exported variables take precedence over the file for both the Python commands and docker compose, so later .env edits would be silently ignored in that shell.

```
( set -a; source .env; set +a      # subshell keeps .env values out of your shell
body='{ "start": "1970-01-01T00:00:00Z", "stop": "2100-01-01T00:00:00Z", '
body="$body"'"predicate": "_measurement=\"telemetry\"" }'
curl -X POST -H "Authorization: Token $INFLUX_TOKEN" \
-H "Content-Type: application/json" -d "$body" \
  "localhost:8086/api/v2/delete?org=$INFLUX_ORG&bucket=$INFLUX_BUCKET" )
pvdX-decode --redo
```

8.6 Connect the mission-control web app

The pvdX-mission-control backend reads telemetry:battery, elevation, temperature, signal_rssi and uptime_seconds from Redis and reports stale when a key is missing. Point its Redis client at the Redis this service publishes to, delete its fake telemetry loop, and set TELEMETRY_ALIASES here so those names map to real decoded fields (.env.example has the CroCube mapping, section 7.2). Its GET /telemetry then returns real values already converted by this service (battery in V, temperature in degC, signal_rssi in dBm, uptime in s; units in telemetry:_meta), so it must not convert them again. It still reports stale because nothing publishes elevation, and its signal_rssi is the spacecraft receiver's RSSI, not the downlink strength at our station. Its fake telemetry loop writes the same telemetry:* keys, so it must not run against this Redis. The frontend's telemetry panel could instead read this service's GET /telemetry (same shape, plus units) once its hardcoded backend URL is changed; add its origin to API_CORS_ORIGINS.

8.7 Push frames from the ground station

Install this package on the ground station computer, set PUSH_URL (the service), PUSH_STATION (this station's name), PUSH_TOKEN (equal to the service's INGEST_TOKEN) and NORAD_CAT_ID in its .env, then:

```

pvdx-push frame1.bin frame2.bin                # one raw frame per file; time = mtime
pvdx-push --time 2026-09-27T14:03:05Z --frequency 436500000 --meta pass=12 frame.bin
pvdx-push --watch 5 /var/lib/gnuradio/frames/  # keep sending new files from a directory
printf '2026-09-27T14:03:05Z 86a2...\n' | pvdx-push --stdin    # "[ISO-time] hex" lines

```

Any HTTP client can do the same: POST /ingest/frames with header X-Ingest-Token and the JSON body {norad_cat_id, station, frames: [{raw (base64), received_at, frequency?, rssi?, meta?}]]; at most 1000 frames per request and 64 KiB per frame. With INGEST_TOKEN set, a request without the right token gets 401 before its body is read. Pushed frames are keyed by (satellite, station, receive second, sha256), so resending is harmless. They are decoded within about a second and show up in the API, Redis and InfluxDB with source = groundstation and the station name. A pushed frame with the same bytes as a SatNOGS frame up to 45 s apart is a copy of the same transmission (section 7.3).

8.8 Upgrade

From 0.1. Install the new version (`uv pip install -e ".[dev]"`), add the new keys from `.env.example` to `.env` (all have working defaults) and start `pvdx-serve`. The state database is migrated in place on first open, from schema 1 to 2 and on to 3 as below (a backup copy of `data/state.db` first is cheap); already-decoded frames are queued again so their decoded fields get stored. The 0.2 writer adds a source tag, so the rewritten InfluxDB points become new series next to the 0.1 points instead of replacing them: delete the measurement first (8.5) to avoid double counts. Downgrading afterwards is not supported; the old code refuses a newer schema.

From 0.2.0 (schema 2 to 3, duplicate receptions). The migration is one-way, so back up first:

```

# stop pvdx-serve (Ctrl-C), or the pvdx container:
# docker compose --profile cloud stop pvdx
sqlite3 data/state.db ".backup state-v2.db" # and the InfluxDB volume (8.9)
uv pip install -e ".[dev]"                # the new code
pvdx-ingest --stats                        # first open: migrates to schema 3
# delete the telemetry measurement (8.5), then rewrite every point with the tag:
pvdx-decode --redo
# start again (or: docker compose --profile cloud up -d --build):
pvdx-serve

```

The migration adds `family` and `primary_frame_id` to every frame and assigns all primaries in one transaction (0.15 s for a 15,926-frame CroCube database: 7,406 transmissions, 8,520 copies). It does not recompute `latest_values`: a field's latest value may still name a frame that is now a copy (the same value, with the copy's frame id, time, source and station) until a newer primary replaces it. InfluxDB has to be rebuilt because the 0.2.0 points have no `primary` tag, so they would stay as separate series next to the rewritten points: hidden under `Receptions = Primary only / Copies only` and counted twice under `All`. For engineering units in Redis and `/telemetry`, copy the new `TELEMETRY_ALIASES` line from `.env.example`; a plain `alias=field` list keeps serving raw values.

8.9 Back up

Copy `data/state.db` (with `sqlite3 data/state.db ".backup backup.db"` while the pipeline runs) and, if wanted, the Docker volumes `bse_code_influxdb-data` and `bse_code_grafana-data`. InfluxDB can always be rebuilt from `state.db` with `pvdx-decode --redo`.

9. Swapping in PVDX

- Set `NORAD_CAT_ID` to PVDX's catalog number (or the temporary id SatNOGS assigns after launch) and `DECODER=pvdx` in `.env`.
- Implement `PvdxDecoder.decode` in `pvdx_ground/decode/pvdx.py`: parse the USLP transfer frame, extract the Space Packet(s), map each APID to fields, and return a flat `dict[str, float | int | str]` of plain Python scalars (other types, such as numpy integers, lists or nested structs, are dropped). The stub currently raises `decodeError` and lists the steps as TODO comments. If PVDX instead gets a Kaitai struct merged into `satnogs-decoders`, `DECODER=satnogs:pvdx` needs no code.
- Record a real PVDX frame under `tests/fixtures/frames/` and add a decode test next to the CroCube ones in `tests/test_decode_storage.py`.
- Set `TELEMETRY_ALIASES` so the web app's names map to PVDX field names; fields the decoder already returns in engineering units need only the `:unit`, no scale or offset.
- Grafana needs no change; tune the `voltage_fields` / `temperature_fields` regexes if desired. The push path is decoder-agnostic: the ground station sends whatever bytes it demodulated.

These steps are enough only for single-frame, real-time beacons. The interface is `decode(raw)` returning one flat dict with no frame context; every frame becomes one point at its receive time, non-scalar fields are dropped, and frames are decoded in row-id order with sources and stations interleaved, duplicate receptions included (grouping looks only at bytes and time, section 7.3). Interface work is needed first for:

- **Multi-packet frames**: two packets with the same fields in one frame collapse into one value; the decoder has to return a list of records, each with its own timestamp and APID/VCID.
- **On-board timestamps** (stored or playback telemetry): the point time is the receive time, so the on-board time can only become a field.
- **Stateful reassembly** of packets that span frames: decoder state lives only in memory, is lost on restart, and the decoder is not told the source, station or push metadata to keep streams apart.
- **CFDP files and photos**: there is no file intake, storage or endpoint (phase 2, section 13).

10. Troubleshooting

Symptom	Cause and fix
ERROR ... rejected SATNOGS_API_TOKEN (401 Invalid token)	The token is a SatNOGS DB token or is wrong. Ingest continues anonymously at 60 req/h. Create a Network API key (section 2.3).
WARNING ... HTTP 429; retry n/6 in Ns	Throttled. The client honours Retry-After and continues; nothing to do. If it repeats, another process on the same IP is also querying SatNOGS, or <code>INGEST_MIN_INTERVAL</code> was lowered.
configuration error: NORAD_CAT_ID is not set	<code>.env</code> is missing, not in the working directory, or the value is empty. Run from the repository root or pass <code>--env</code> / <code>--norad</code> . Exit code 2.
configuration error: no decoder configured / has no struct named ...	<code>DECODER</code> is empty or misspelt. Set <code>DECODER=satnogs:<struct></code> (name as on <code>db.satnogs.org</code> , case-insensitive) or <code>pvdx</code> , or pass <code>--decoder</code> . Exit code 2.

Symptom	Cause and fix
configuration error: INFLUX_URL must start with http:// or https:// / REDIS_URL must start with redis://, rediss:// or unix:// / INGEST_MIN_INTERVAL must be >= 15	A value in .env (or a flag) has the wrong scheme or is below its minimum; the URL itself is not repeated in the message. Fix it and start again. Exit code 2.
configuration error: TELEMETRY_ALIASES entry ...	A malformed alias entry: not alias=field[*scale][[+ -]offset][:unit] (a missing =, an empty unit after the colon, ...), or a scale of 0. See section 7.2. Exit code 2.
HTTP 400 ... (from the API)	--status and --since are validated locally (exit 2 with a 'configuration error' line), so a real HTTP 400 points at SATNOGS_NETWORK_URL or an API change. A rejected saved cursor is handled automatically (next row).
saved cursor for sweep N rejected ... aborting	SatNOGS no longer accepts an old cursor. The sweep is dropped (that run exits 1; under --poll the next poll retries) and the next run starts a fresh sweep from watermark minus overlap (from INGEST_START if no sweep has completed yet). Just run again.
sqlite3.OperationalError: database is locked	Another connection held the write lock for over 30 s. pvdx-ingest and pvdx-decode only use short per-observation/per-frame transactions, so look for a manual sqlite3 session left inside a transaction, or a state.db on a network share. Normal ingest + decode + API concurrency is fine.
InfluxDB not reachable ... / InfluxDB rejected the credentials (401) / bucket 'telemetry' does not exist	Startup check of pvdx-decode. Stack down or still starting: docker compose up -d and wait for (healthy). 401: INFLUX_TOKEN/INFLUX_ORG in .env differ from what the InfluxDB volume was initialised with. Missing bucket: INFLUX_BUCKET/INFLUX_ORG differ. Fix .env or reset the volumes (8.5).
INFLUX_TOKEN is not set	Add INFLUX_TOKEN to .env (must match what the InfluxDB container was initialised with; otherwise reset the volumes).
Dashboard shows No data everywhere	No points yet: run pvdx-decode. If the NORAD ID dropdown is empty, no point is newer than 30 days (the variable queries use a fixed 30-day look-back, independent of the time picker): ingest and decode newer data, or edit start: -30d in the variable queries. If the dropdown is populated but panels are empty, widen the time range (default: last 7 days).
Dashboard variables show a warning triangle	Grafana cannot query InfluxDB. Check INFLUX_TOKEN in .env matches what the InfluxDB volume was initialised with, then docker compose up -d grafana to recreate the container (docker compose restart does not re-read .env). The datasource is provisioned read-only, so it cannot be fixed from the Grafana UI.
docker: unknown command: docker compose	Docker Desktop is installed but was never started, so the Compose plugin is not linked. Start Docker Desktop once, then retry.
Grafana asks to update the password on every login	Expected with the default admin password; click Skip or set a password (it is stored in the grafana-data volume).
Many frames marked error with UnicodeDecodeError / EOFError	Non-telemetry frames (image chunks, digipeater packets, truncated captures). Expected; only beacons decode.
ERROR frame N: <decoder>: unexpected <exception>: ...	The decoder raised something other than a normal rejection: a decoder bug. That frame is marked error with the same text in decode_error and decoding goes on (one traceback per pass). Fix the decoder, then pvdx-decode --redo.

Symptom	Cause and fix
"Frames in range" is about half of "Receptions in range"	Expected: most transmissions are received more than once (several stations, gr-satellites_gN files). "Frames in range" counts primaries by default; set the Receptions variable to All to count every reception (section 7.3).
API returns 400 or 422	400: a since/until that is not ISO-8601 or relative or lies outside the years 1-9999, an invalid field name, or a /telemetry endpoint called without norad while NORAD_CAT_ID is unset and the database does not hold exactly one satellite. 422: a value out of range (ids and ground_station at least 1, norad 1 to 999 999 999, field 1 to 200 characters, limit/offset bounds, source/decode_status/primary/copies values) or a malformed push body. See section 6.2.
pvdX-push: HTTP 401 missing or invalid X-Ingest-Token	PUSH_TOKEN on the station differs from INGEST_TOKEN on the service (or the service has none and the client sends one is fine; the reverse is not). Align the two.
WARNING INGEST_TOKEN is not set: POST /ingest/frames accepts frames from anyone	Fine on a laptop. Set INGEST_TOKEN before exposing the service beyond localhost.
Redis publisher: Redis at ... not reachable / could not publish latest telemetry	Redis is down or not reachable at REDIS_URL. Decoding continues; once Redis answers, keys are republished by the next decode pass that writes at least one decoded frame to InfluxDB, or on a restart. docker compose up -d redis.
GET /health returns degraded with workers.decode = stopped	The decode (or ingest) thread hit an unrecoverable error; see the log line 'worker stopped after an unrecoverable error' above it. The process and the API keep running (HTTP 200), so no restart policy restarts it: fix the cause and restart pvdX-serve.
Grafana, the API, InfluxDB or Redis unreachable from another machine	Expected: compose publishes every port on 127.0.0.1, and a host-run pvdX-serve listens on API_HOST (default 127.0.0.1). Use a TLS reverse proxy on the host, or set GRAFANA_BIND / API_BIND for the containers or API_HOST=0.0.0.0 for a host-run pvdX-serve (section 3). InfluxDB and Redis stay on loopback.
[Errno 48] address already in use (uvicorn)	Another pvdX-serve (or the pvdX container) already listens on API_PORT. Stop it or pass --port.
docker: RWLayer of container ... is unexpectedly nil	Docker Desktop lost track of an old container. docker rm -f pvdX-influxdb pvdX-grafana, then docker compose up -d; the data volumes are untouched.
GET /telemetry shows stale: true although frames arrive	The newest decoded frame is older than TELEMETRY_STALE_AFTER, or the pushed received_at times are wrong (they must be UTC; naive times are taken as UTC). Check /telemetry/latest newest_frame_time.

11. Maintenance and tests

```

pytest                                # 151 tests on recorded pages + real frames, offline
uv pip install -U satnogs-decoders    # newer Kaitai structs (after an upstream fix)

```

Test fixtures live in tests/fixtures/: two consecutive observation pages with their real Link headers, eleven frame files from those pages, and four real CroCube frames with their expected decode. The API is tested with FastAPI's test client, Redis with an in-memory stand-in, and pvdX-push against a mock service; the decode loop's wake-up on push and its behaviour during a Redis outage are covered too, and so are both schema migrations, duplicate grouping (windows, anchoring, sticky primaries, the primary tag, history

copies, the /frames filter, the dashboard's Receptions variable), alias unit conversion, concurrent API requests, out-of-range parameters, the push token checked before the body, decoder crashes, URL scheme and interval checks, and SIGTERM shutdown. To refresh them, record new pages with `curl`, update `manifest.json`, the `.link` files and the NORAD/SINCE constants in `tests/conftest.py`, and revisit the assertions that encode the recorded set's shape (two pages of 25 plus an empty last page, 11 frames of which 2 on page 1, one page-1 observation without demodata) in `tests/test_worker.py` and `tests/test_client.py`.

12. Verified SatNOGS API facts

These were checked against the live API and the satnogs-network source on 2026-09-26 and differ from older documentation:

- The satellite filter is `norad_cat_id`; `satellite__norad_cat_id` is silently ignored and returns every satellite.
- `status` is a string (failed, bad, unknown, future, good); integers return HTTP 400.
- The observations endpoint returns a bare JSON list of 25 items; the next page is only advertised in the `Link: <url>; rel="next"` header. Ordering is newest first.
- `start` means `start >= value`, `end` means `end <= value`; `start__lt`, `end__gt` and comma-separated `observation_id` also exist.
- Throttle: 60 list requests per hour anonymous, 240 with a Network token; 429 responses carry `Retry-After`. Frame files on object storage are not throttled.
- The OpenAPI schema URL in the original brief (`network.satnogs.org/api/schema/`) is a 404; SatNOGS DB publishes one at `db.satnogs.org/api/schema/?format=json`.

13. Known gaps

- Backfill speed is bounded by the SatNOGS throttle (about 1 500 observations per hour anonymously).
- Observations vetted good later than the overlap window are missed. Moving `INGEST_START` back does not help once a watermark exists (every sweep starts at `max(INGEST_START, watermark minus overlap)`); run one sweep with a larger overlap, e.g. `pvdX-ingest --overlap-hours 720`, or delete the satellite's row in the `watermarks` table.
- No CRC or range validation: Kaitai structs are permissive (garbage bytes can decode into nonsense values), the push endpoint checks only base64, emptiness and size, and the PVDX FECF check is a TODO.
- `decode_status = error` mixes non-telemetry frames (image chunks, digipeater packets) with real parse failures, decoder crashes (unexpected in `decode_error`) and InfluxDB rejections, so it cannot flag a decoder regression; errored frames are only retried by a full `pvdX-decode --redo`.
- Duplicate grouping looks only at identical bytes and time: a copy whose time is more than 45 s from the primary's (a late station clock plus a `_gN` copy's early stamp can add up to that), or with a bit error, stays a transmission of its own, and two genuine transmissions with identical bytes at most 45 s apart (30 s within one observation) can merge. Assignments are sticky and times are not corrected, so a `_gN` copy assigned before its native twin keeps its primary and its 0-16 s early time, as do transmissions heard only as `_gN`. Per-station views count receptions ("Frames by station"), or with "Primary only" just the transmissions a station supplied the primary for.
- All numbers are written as floats; enum-like fields appear as strings and are not plottable.
- One satellite per `pvdX-serve` process, and the Redis keys are not prefixed by satellite; pushed frames for another satellite are stored but not decoded until that id is configured.
- The PVDX decoder is a stub until the USLP/SPP format is frozen.
- No authentication on the API (left to the web app layer); only the push endpoint has a shared secret. Put the service behind the backend or a TLS reverse proxy before exposing it.
- Not built (phase 2): scheduling observations on the SatNOGS Network, CFDP reassembly of downlinked files/photos, and the commanding/uplink path.

Confirmed open issues that await a decision before they are fixed:

- **Freshness:** the Redis TTL runs from publish time, so restarts and backfills make old values look current, and freshness is per satellite, not per field: one fresh beacon type re-arms every key and clears stale in `/telemetry` while other fields are hours old.
- **Web-app contract:** units are handled (the aliases convert in Redis and `/telemetry` and name their units, so the team backend must not convert again). Still open: nothing publishes `elevation` (its meaning is undecided; the team backend always reports stale), the team's fake telemetry loop writes the same `telemetry:*` keys (key namespace undecided), and freshness (above).
- **Future-dated pushes** pin latest values: a `received_at` in the future (up to InfluxDB's year 2262 limit) is accepted and freezes those fields in the API and Redis until a genuinely newer frame arrives; `--redo` does not clear it.
- **Partial push batches:** a bad frame later in a batch (invalid base64, empty, over 64 KiB, `received_at` out of range) returns 400/413 after the earlier frames were stored, with no ids reported, and those are only decoded at the next `DECODE_POLL`. Resending is safe.

- **InfluxDB is a hard gate:** while it is down nothing new reaches `latest_values`, Redis or the API (it catches up afterwards); a point it refuses marks the whole frame error and drops all of its fields.
- **GET /health always returns HTTP 200**, also when degraded, so the Docker HEALTHCHECK passes with a dead worker or InfluxDB down, and a dead worker does not end the process, so no restart policy fires; the body has no progress signals (last successful pass, backlog).
- **Ingest:** a frame whose download fails on 5 runs is abandoned until `pvdX-ingest --retry-failed` (not shown in `/health`); a saved cursor that keeps failing with anything but 400/404 leaves its sweep open, so new passes are never fetched; observations re-vetted from good to bad keep their status and frames.
- **pvdX-push:** `--watch` exits on the first failed or rejected push, an unreadable file or a file over 64 KiB (which then blocks the files after it on every restart), and sends files still being written as partial frames; `--stdin` sends nothing until EOF and loses the whole buffer on one bad line or an outage at EOF.
- **BSE station not selectable in Grafana:** pushed points have no `ground_station` tag (section 6.1); the identifier for our own station is undecided.
- **SiDS frames not ingested:** only the SatNOGS Network is read, not frames that independent stations upload to SatNOGS DB, although for CroCube they carry a sizeable share of the decodable telemetry.